Economics S5040: Dynamic Stochastic Modeling
Lecture 8 — Solving Discrete Dynamic Programs

Eric Fisher

Fall 2026

1 The Student Problem

We now study a small stochastic dynamic program that we can understand almost entirely by hand.

A student can be in one of three health states:

$$X = \{\text{Sick}, \text{Okay}, \text{Great}\}.$$

She has only two possible controls:

$$U = \{\text{TV}, \text{Study}\}.$$

The reward function depends upon the state and the controk

$$r(x,u) = \begin{array}{c|cc} & \text{TV} & \text{Study} \\ \hline \text{Sick} & 1 & -1 \\ \text{Okay} & 2 & 1 \\ \text{Great} & 8 & 5 \end{array}.$$

Thus, for example,

$$r(\text{Great}, \text{TV}) = 8.$$

The student always likes to watch TV, but studying really hurts when she is sick.

The action also affects tomorrow's health. If the student watches TV,

$$M_{\text{TV}} = \begin{pmatrix} 0.5 & 0.5 & 0 \\ 0.1 & 0.4 & 0.5 \\ 0 & 0.4 & 0.6 \end{pmatrix}.$$

If she studies,

$$M_{\text{Study}} = \begin{pmatrix} 0.9 & 0.1 & 0 \\ 0.3 & 0.6 & 0.1 \\ 0.3 & 0.6 & 0.1 \end{pmatrix}.$$

Rows are today's health state and columns are tomorrow's health state.

For example,

$$M_{\text{TV}}(Sick, Okay) = 0.5$$

means that a sick student who watches TV has a 50 percent probability of being Okay tomorrow.

This is the first important feature:

the transition matrix depends on the control.

This is an illustration of the idea that $x' \sim g(x,u)$; what one does today will influence what happens tomorrow. For example, if the student is sick and she chooses to study, then she will likely be sick again tomorrow. But watching TV gives her a 50-50 chance of being okay tomorrow.

2 The Bellman Equation

Let

$$V_S, \qquad V_{OK}, \qquad V_G$$

be the values of beginning the period Sick, Okay, and Great.

For state x and action u, the value of choosing u today and behaving optimally thereafter is

$$r(x,u) + \beta \sum_{x'} M_u(x,x')V(x').$$

Hence

$$\boxed{V(x) = \max_{u \in \{\text{TV,Study}\}} \left\{ r(x,u) + \beta \sum_{x'} M_u(x,x')V(x') \right\}.}$$

For example, in the Sick state,

$$V_S = \max\left\{1 + \beta(.5V_S + .5V_{OK}),\ -1 + \beta(.9V_S + .1V_{OK})\right\}.$$

The Bellman equation is an implicit definition of V: the unknown value function appears on both sides. In most interesting models we cannot simply solve this equation analytically. We need a computational method.

Today we study two.

3 Method 1: Howard's Policy Improvement

Howard's method starts with a guess about the *policy*, not the value function.

Suppose we guess

$$h_0(x) = \text{TV} \qquad \text{for every state } x.$$

To illustrate policy evaluation, the Howard calculation in the original notebook uses the reward vector

$$r_{h_0} = \begin{pmatrix} 1 \\ 2 \\ 8 \end{pmatrix}$$

and $\beta = 0.5$.

Under the guessed policy, the relevant transition matrix is

$$P_{h_0} = M_{\text{TV}}.$$

If the student follows this policy forever, its value satisfies

$$V_{h_0} = r_{h_0} + \beta P_{h_0} V_{h_0}.$$

Rearranging,

$$(I - \beta P_{h_0})V_{h_0} = r_{h_0}.$$

Thus

$$\boxed{V_{h_0} = (I - \beta P_{h_0})^{-1} r_{h_0}.}$$

Numerically,

$$V_{h_0} \approx \begin{pmatrix} 3.64 \\ 6.92 \\ 13.40 \end{pmatrix}.$$

This step is called **policy evaluation**.

We have not proved that TV is optimal. We have only answered:

What would each state be worth if the student watched TV forever?

The box equation is three linear equations in three unknowns–the values of being sick, okay, and great–assuming that one always wtahces TV. Such a system can be solved by hand.

4 Policy Improvement

Now ask whether the student wants to deviate from the guessed policy.

For a Sick student, TV gives

$$1 + 0.5(.5(3.64) + .5(6.92)) \approx 3.64.$$

The Study reward is $r(Sick, Study) = -1$. So Study gives

$$-1 + 0.5(.9(3.64) + .1(6.92)) \approx 0.98.$$

Thus–under our initial guessed policy– TV is better when the student is Sick. So far our guess is verified.

What should an okay student do?

For an Okay student, TV gives

$$2 + 0.5(0.1(3.64) + .4(6.92) + 0.5(13.40)) \approx 6.92$$

For an Okay student, Study gives

$$1 + 0.5(0.3(3.64) + 0.6(6.92) + 0.1(13.40)) \approx 4.29$$

Again, our guess is so far verified because an okay student does better by watching TV, if the value function is calculated according to our original guessed policy.

What about a student who is feeling great?

TV gives her

$$8 + 0.5(0.0(3.64) + 0.4(6.92) + 0.6(13.40)) \approx 13.40$$

And Study gives her

$$5 + 0.5(0.3(3.64) + 0.6(6.92) + 0.1(13.40)) \approx 8.29$$

So she too should watch TV.

We got lucky here. Our initial guess $h_0(x)$ was right. In this problem, there are 2^3 possible policies, and we picked the correct one right off the bat by wild chance.

More generally, define the action value

$$Q(x, u; V) = r(x, u) + \beta \sum_{x'} M_u(x, x')V(x').$$

Given the value of a guessed policy, improve the policy by choosing

$$\boxed{h_1(x) = \arg\max_u Q(x, u; V_{h_0}).}$$

Howard's method is therefore:

1. Guess a policy h_0.
2. **Evaluate** that policy.
3. **Improve** it by finding the best action in each state.
4. If the policy does not change, stop.
5. Otherwise evaluate the new policy and repeat.

The conceptual distinction is crucial:

$$\boxed{\text{policy evaluation}} \qquad \text{versus} \qquad \boxed{\text{policy improvement}}.$$

5 Policy Evaluation in Python

For a fixed policy, the computer solves a system of linear equations.

```
import numpy as np

reward = np.array([[1, -1],
                   [2, 1],
                   [8, 5]])

M_tv = np.array([[0.5, 0.5, 0.0],
                 [0.1, 0.4, 0.5],
                 [0.0, 0.4, 0.6]])

M_study = np.array([[0.9, 0.1, 0.0],
                    [0.3, 0.6, 0.1],
                    [0.3, 0.6, 0.1]])

transition = [M_tv, M_study]
beta = 0.5

policy = np.array([0, 0, 0]) # TV in every state

r_h = reward[np.arange(3), policy]
P_h = np.vstack([
    transition[policy[x]][x]
    for x in range(3)
])

V_h = np.linalg.solve(
    np.eye(3) - beta * P_h,
    r_h
)
```

The economics is more important than the syntax. Once a policy is fixed, the optimization disappears and we are simply valuing a Markov reward process.

6 Method 2: Value Function Iteration

The second method starts with a guess about the *value function*.

Let

$$V_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

Use the Bellman equation to construct a new guess:

$$V_1(x) = \max_u \left\{ r(x,u) + \beta \sum_{x'} M_u(x,x') V_0(x') \right\}.$$

Because $V_0 = 0$, the first iteration ignores future rewards:

$$V_1(x) = \max_u r(x,u).$$

Then use V_1 to construct V_2, and continue.

Define the **Bellman operator** T by

$$\boxed{(TV)(x) = \max_u \left\{ r(x,u) + \beta \sum_{x'} M_u(x,x') V(x') \right\}.}$$

Value function iteration is simply

$$\boxed{V_{n+1} = TV_n.}$$

Each application of T brings future rewards into the calculation.

7 One Bellman Step in Python

The key computation is short:

```
def bellman(V, R, M, beta):
    n = R.shape[0]
    k = R.shape[1]

    next_V = np.empty(n)

    for x in range(n):
        action_values = np.empty(k)

        for u in range(k):
            expected_V = M[u][x] @ V
            action_values[u] = (
                R[x, u] + beta * expected_V
            )

        next_V[x] = np.max(action_values)

    return next_V
```

Focus on

`M[u][x] @ V`.

The row `M[u][x]` is the probability distribution over tomorrow's states conditional on current state x and action u.

Therefore

$$\texttt{M[u][x] @ V} = \sum_{x'} M_u(x, x')V(x') = E[V(x') \mid x, u].$$

The code is just the Bellman equation translated into Python.

8 Iterating Until the Value Function Stops Changing

We repeatedly apply the Bellman operator:

```
V = np.zeros(3)

for i in range(500):
    next_V = bellman(V, reward, transition, beta)

    if np.allclose(next_V, V):
        break

    V = next_V
```

When successive value functions are sufficiently close, we stop.

Once we have the value function, the optimal policy is obtained from

$$h(x) = \arg\max_u \left\{ r(x, u) + \beta \sum_{x'} M_u(x, x')V(x') \right\}.$$

In Python, `np.argmax` returns the index of the maximizing action.

9 Howard Versus Value Function Iteration

Both methods solve the same dynamic program, but they approach it differently.

Howard policy improvement	**Value function iteration**
Start with a policy.	Start with a value function.
Evaluate the policy.	Apply the Bellman operator.
Improve the policy.	Repeat the Bellman operator.
Stop when the policy no longer changes.	Stop when the value function no longer changes appreciably.

For this small problem, both methods are transparent enough to understand by hand. In larger problems, the computer performs the same logic on much larger state and control spaces.

10 The Optimal Policy Creates a Markov Chain

There is one final connection to last week's material.

Suppose the optimal policy is $h(x)$. In each state, the policy selects an action, and that action selects a row from the corresponding transition matrix.

Thus the optimal policy generates a new transition matrix P_h:

$$P_h(x, \cdot) = M_{h(x)}(x, \cdot).$$

Once we have solved the dynamic program, we can therefore ask familiar Markov questions:

- How does the distribution of states evolve?
- Does it converge?
- What is the stationary distribution?

This is an important connection:

optimization determines behavior → behavior determines transitions → transitions determine long-run outcomes

11 What to Remember

There are four central ideas today:

1. In a stochastic dynamic program, the action can affect both today's reward and tomorrow's transition probabilities.
2. Howard's method alternates between **policy evaluation** and **policy improvement**.
3. Value function iteration repeatedly applies the Bellman operator:

$$V_{n+1} = TV_n.$$

4. Once the optimal policy is known, it induces a Markov transition matrix for the optimally controlled system.

The main technique of the course is now in place:

$$\boxed{V(x) = \max_u \left\{ r(x, u) + \beta E[V(x') \mid x, u] \right\}.}$$

In the next application, the states and choices will have direct economic interpretations, but the underlying logic will be the same.